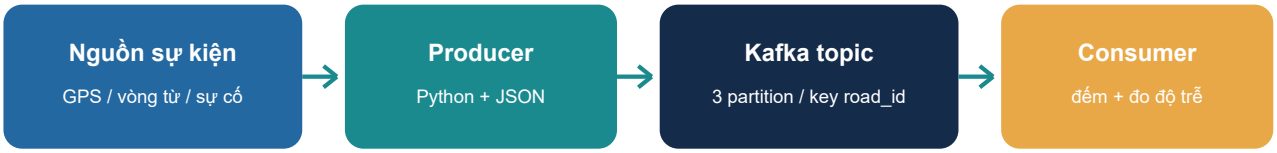
TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN HỌC PHẦN

NHẬP MÔN DỮ LIỆU LỚN

Thiết kế và triển khai quy trình Sources -> Ingestion cho dữ liệu giao thông



Ranh giới bài toán: nhận sự kiện và ghi bền vững vào topic; downstream chỉ quan sát để kiểm chứng.

Lớp học phần	Bổ sung theo thông tin lớp
Nhóm / thành viên	Bổ sung tên, mã số sinh viên và phân công
Năm học	2026

Báo cáo kỹ thuật về pipeline tiếp nhận sự kiện giao thông thời gian thực bằng Apache Kafka

Tóm tắt

Đồ án triển khai quy trình Sources -> Ingestion cho bài toán tiếp nhận liên tục sự kiện giao thông tại Thành phố Hồ Chí Minh. Nguồn Python mô phỏng ba loại dữ liệu có cấu trúc khác nhau: vị trí GPS phương tiện, số đo cảm biến vòng từ và báo cáo sự cố từ trung tâm điều phối. Các sự kiện được đóng gói thành JSON với envelope chung và payload theo loại, định tuyến theo road_id vào topic traffic-events gồm ba partition. Apache Kafka chạy ở chế độ KRaft một broker trong Docker Compose; producer dùng xác nhận broker, retry và nén gzip. Consumer tham gia consumer group trước khi phát, đếm dữ liệu nhận, tính throughput và độ trễ event-to-consume. Thử nghiệm thực tế gửi tổng cộng 15.000 sự kiện ở các mức 100, 500 và 1.000 sự kiện/giây. Cả ba lượt đều được broker xác nhận đủ và consumer đọc đủ; throughput trung bình lần lượt là 99,3; 482,0; và 932,1 sự kiện/giây. p95 độ trễ đo được lần lượt là 18,1; 19,9; và 20,7 ms. Mỗi lượt cũng xác nhận event_id duy nhất, không có sequence trùng/khuyết và đủ ba event_type. Kết quả chứng minh luồng demo hoạt động liên tục ở các tải đã thử, nhưng không đại diện cho cluster production hoặc khả năng chịu lỗi nhiều broker.

Từ khóa

Big Data, ingestion, Apache Kafka, streaming, producer, consumer, partition, throughput, latency.

Mục lục

Tóm tắt 2

Từ khóa . . . 2

Mục lục 3

1. Giới thiệu 5

1.1 Bối cảnh . . . 5

1.2 Bài toán và câu hỏi kỹ thuật . . . 5

1.3 Mục tiêu và phạm vi . . . 5

1.4 Đóng góp . . . 5

2. Cơ sở lý thuyết và công nghệ 5

2.1 Đặc trưng Big Data liên quan . . . 5

2.2 Mô hình streaming . . . 6

2.3 Các khái niệm Kafka sử dụng . . . 6

2.4 Lý do chọn công nghệ . . . 6

3. Dữ liệu và yêu cầu 6

3.1 Nguồn và cách tạo dữ liệu . . . 6

3.2 Hợp đồng dữ liệu . . . 7

3.3 Các payload theo loại . . . 7

3.4 Ví dụ bản ghi . . . 7

3.5 Input và output . . . 7

4. Thiết kế hệ thống 8

4.1 Sơ đồ luồng dữ liệu . . . 8

4.2 Trách nhiệm thành phần . . . 8

4.3 Chính sách topic và partition . . . 8

4.4 Độ bền và xử lý lỗi . . . 9

5. Cài đặt và triển khai 9

5.1 Cấu trúc mã nguồn . . . 9

5.2 Cấu hình và khởi động . . . 9

5.3 Producer . . . 9

5.4 Consumer và đo lường . . . 10

5.5 Khả năng tái lập . . . 10

6. Thực nghiệm và đánh giá 10

6.1 Môi trường . . . 10

6.2 Thiết kế kịch bản . . . 10

6.3 Kết quả định lượng . . . 10

6.4 Đánh giá theo yêu cầu . . . 12

7. Kết quả demo 12

7.1 Luồng demo 12
7.2 Minh chứng nộp 12
8. Thảo luận **12**
8.1 Điểm đạt được 12
8.2 Hạn chế 12
8.3 Hướng cải tiến trong pipeline tiếp theo 13
8.4 Phân công nhóm 13
9. Kết luận **13**
Phụ lục A. Đối chiếu yêu cầu đồ án **14**
Phụ lục B. Tài liệu tham khảo **14**
Phụ lục C. Công cụ AI và xác nhận kỹ thuật **14**

1. Giới thiệu

1.1 Bối cảnh

Các hệ thống giao thông tạo ra sự kiện liên tục từ thiết bị GPS, cảm biến mặt đường và báo cáo của trung tâm điều phối. Nếu chờ thu thập thành một tập tệp rồi mới xử lý, việc cung cấp dữ liệu mới cho các giai đoạn phân tích phía sau sẽ bị trễ. Vì vậy, giai đoạn ingestion cần tiếp nhận sự kiện theo thời gian, chuẩn hóa giao diện message, ghi vào một hàng đợi bền vững và cho phép consumer đọc độc lập.

Trong kiến trúc Big Data, ingestion là ranh giới giữa nguồn phát và tầng lưu trữ/processing. Kafka phù hợp để làm broker sự kiện: producer phát record vào topic, broker phân vùng record, consumer group đọc theo offset. Tài liệu học phần mô tả streaming là chuỗi sự kiện không có điểm kết thúc tự nhiên và nhấn mạnh latency, throughput, event time, topic, partition và consumer group [1, 2].

1.2 Bài toán và câu hỏi kỹ thuật

Bài toán là tiếp nhận liên tục dữ liệu giao thông giả lập từ nhiều loại nguồn vào một topic Kafka để các thành phần sau có thể lấy lại theo offset. Bài toán kỹ thuật cần trả lời: (1) producer có thể duy trì tốc độ phát mục tiêu không; (2) broker ACK có đối chiếu được với số consumer nhận không; (3) khóa phân vùng nào giữ sự kiện cùng tuyến đường ở cùng partition; và (4) độ trễ thay đổi thế nào khi tăng tốc độ phát.

1.3 Mục tiêu và phạm vi

- Triển khai đúng quy trình con Sources -> Ingestion trong danh sách đề bài [1].
- Sinh và đưa sự kiện JSON vào Kafka topic có cấu hình partition xác định.
- Ghi log mẫu sự kiện, tốc độ phát, số ACK, số nhận và độ trễ.
- Cung cấp lệnh chạy tái lập và log thực nghiệm để kiểm tra độc lập.

Ranh giới kết thúc tại topic và consumer quan sát phục vụ minh chứng. Không triển khai lưu Data Lake, xử lý Spark, phân tích nghiệp vụ hoặc dashboard; các nội dung đó thuộc các giai đoạn khác của pipeline [1].

1.4 Đóng góp

Sản phẩm gồm Docker Compose cho Kafka KRaft một node, producer Python nhiều loại nguồn mô phỏng, script tạo topic và điều phối benchmark, consumer đo số liệu, cùng log thử tải. Thiết kế nêu rõ schema message, chiến lược key, đảm bảo ACK, cách tái lập và giới hạn của kết quả.

2. Cơ sở lý thuyết và công nghệ

2.1 Đặc trưng Big Data liên quan

Đặc trưng	Thể hiện trong bài toán	Minh chứng / giới hạn
Velocity	Sự kiện phát liên tục; tốc độ mục tiêu tăng theo ba kịch bản.	100/500/1.000 sự kiện/s được duy trì trong các đợt 5.000 sự kiện.
Variety	GPS xe, cảm biến vòng từ, báo cáo sự cố; payload và trường khác nhau.	Ba event_type cùng được gửi vào một topic JSON bán cấu trúc.
Volume	Dòng sự kiện có thể tích lũy theo thời gian và được nạp theo lô tải.	Thử 15.000 sự kiện; đây là mô phỏng tải có kiểm soát, không phải tập Big Data cấp TB.

Đồ án thể hiện rõ Velocity và Variety, đồng thời dùng Volume như một kích thước tải thử nghiệm. Một máy và 15.000 bản ghi không tự nó chứng minh quy mô dữ liệu Big Data trong production; lý do dùng kiến trúc ingestion là dữ liệu phát liên tục, có nhiều cấu trúc nguồn và cần tăng tải bằng partition/consumer. Đây là giới hạn cần trình bày trung thực khi bảo vệ.

2.2 Mô hình streaming

Trong batch, đầu vào thường có ranh giới rõ và hệ thống có thể đợi toàn bộ tệp. Trong streaming, event đến liên tục; hệ thống xử lý tăng dần và kết quả có giá trị theo thời gian. Producer gắn event_time theo UTC ISO-8601. Consumer so sánh thời điểm này với thời điểm nhận để ước lượng độ trễ đầu-cuối. Số đo gồm thời gian tạo sự kiện, mạng cục bộ, chờ batch/ACK của producer, broker và poll của consumer; do đó không phải phép đo latency riêng của broker [2].

2.3 Các khái niệm Kafka sử dụng

Khái niệm	Vai trò trong triển khai
Broker	Tiến trình Kafka nhận, lưu log và phục vụ đọc; triển khai demo có một broker.
Topic	traffic-events là luồng logic lưu các event giao thông.
Partition	Topic có ba partition; Kafka phân phối record theo hash của key road_id.
Producer	Tạo JSON, gửi theo key, chờ broker ACK trước khi tính là đã giao.
Consumer group	Consumer có group riêng cho từng lượt thử, bắt đầu ở latest và đếm các event mới.
Offset	Vị trí của record trong partition; dùng để theo dõi tiến độ consumer.

Kafka chính thức hướng dẫn tạo topic, phát/đọc event và kiểm tra partition count trong quickstart [3]. Cấu hình acks=all yêu cầu leader chờ các replica đang đồng bộ; tuy nhiên với replication factor 1 của bài demo chỉ có một replica, vì vậy không có đảm bảo dư thừa khi broker hỏng [4].

2.4 Lý do chọn công nghệ

Công nghệ	Vị trí	Lý do lựa chọn
Apache Kafka 4.1.0	Broker / topic	Bám nội dung streaming của học phần; topic, partition, offset và consumer group thể hiện rõ mô hình ingestion.
Docker Compose	Môi trường broker	Khởi động broker KRaft và volume bằng cấu hình tái lập, không cần cài Java thủ công.
Python 3.14.5	Producer, admin, consumer	Dễ sinh dữ liệu đa loại, nối Kafka và điều phối kịch bản tải.
kafka-python 2.3.3	Client Python	Cung cấp producer, consumer group và admin client trong một dependency.
JSON	Data contract	Đọc được, linh hoạt cho payload khác nhau; thích hợp minh họa variety.

3. Dữ liệu và yêu cầu

3.1 Nguồn và cách tạo dữ liệu

Nguồn là simulator viết bằng Python, không gọi API ngoài và không chứa dữ liệu cá nhân hay GPS thật. Mỗi sự kiện chọn ngẫu nhiên một trong sáu tuyến đường mô phỏng tại TP. Hồ Chí Minh, có tọa độ dao động nhỏ quanh điểm tham chiếu. Tỉ lệ loại sự kiện được đặt 70% GPS phương tiện, 20% cảm biến vòng từ và 10% báo cáo sự cố. Producer có seed để tái lập phần lựa chọn ngẫu nhiên của payload; event_id vẫn là UUID mới ở mỗi lần chạy.

Simulator định kỳ tạo các khoảng tốc độ thấp ở payload GPS để bài toán phía sau có thể nhận diện tình trạng chậm. Việc phân loại này chỉ là dữ liệu demo: chưa áp dụng thuật toán phát hiện ùn tắc và không có ý nghĩa quan sát giao thông thực tế.

3.2 Hợp đồng dữ liệu

Trường	Kiểu	Bắt buộc	Mô tả
event_id	string UUID	Có	Định danh duy nhất của message.
event_time	string ISO-8601 UTC	Có	Thời điểm nguồn tạo sự kiện; dùng đo event-to-consume.
event_type	enum string	Có	gps_position, road_sensor hoặc traffic_incident.
source	string	Có	Tên nguồn mô phỏng tương ứng loại sự kiện.
road_id	string	Có	Khóa Kafka; cố định tuyến phân vùng theo tuyến đường.
district	string	Có	Tên quận/huyện tiếng Việt.
latitude / longitude	number	Có	Vị trí gần tọa độ tham chiếu tuyến đường.
sequence	integer	Có	Số thứ tự trong lượt producer.
payload	object JSON	Có	Trường riêng tùy event_type.

3.3 Các payload theo loại

event_type	Trường payload	Ý nghĩa
gps_position	vehicle_id, speed_kmh, occupancy_pct, traffic_condition	Vị trí và trạng thái xe buýt mô phỏng; tốc độ có khoảng chậm định kỳ.
road_sensor	sensor_id, average_speed_kmh, vehicle_count_1m, rainfall_mm_h	Số đo cảm biến giao thông theo cửa sổ một phút.
traffic_incident	incident_id, incident_type, severity, reported_by	Báo cáo sự cố với loại và mức nghiêm trọng.

3.4 Ví dụ bản ghi

```
{
  "event_id": "5f82...-uuid",
  "event_time": "2026-10-30T21:58:58.241+00:00",
  "event_type": "road_sensor",
  "source": "simulated_loop_sensor",
  "road_id": "D1-NGUYEN-HUE",
  "district": "Qu■n 1",
  "latitude": 10.7744,
  "longitude": 106.7029,
  "sequence": 17,
  "payload": {
    "sensor_id": "LOOP-018",
    "average_speed_kmh": 31.4,
    "vehicle_count_1m": 42,
    "rainfall_mm_h": 0.0
  }
}
```

Ví dụ trên minh họa hợp đồng message; giá trị event_id/tọa độ thay đổi mỗi lần chạy. Log benchmark đính kèm chứa event mẫu thật của lượt thử.

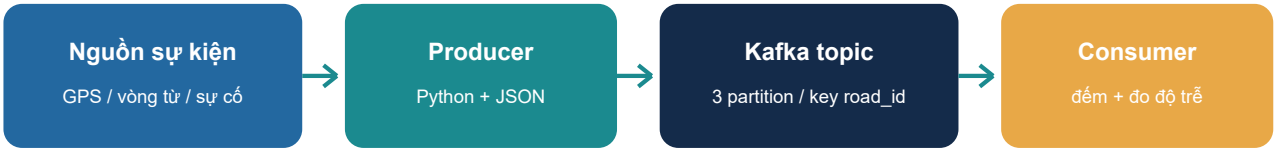
3.5 Input và output

Biên	Đầu vào	Đầu ra
Start	Simulator Python tạo các event JSON theo rate/seed.	Record JSON và key road_id.
End	Kafka broker tiếp nhận record vào topic.	Record được broker ACK và có offset trong partition; consumer monitor xác nhận nhận đủ.

Định dạng đầu ra là Kafka record: topic traffic-events, key dạng UTF-8 road_id và value UTF-8 JSON. Broker không sửa schema; consumer demo giải mã JSON và chỉ dùng event_time để đo latency.

4. Thiết kế hệ thống

4.1 Sơ đồ luồng dữ liệu



Ranh giới bài toán: nhận sự kiện và ghi bền vững vào topic; downstream chỉ quan sát để kiểm chứng.

Luồng một chiều trong phạm vi đồ án là source simulator -> Python producer -> Kafka topic -> consumer monitor. Broker được cấu hình KRaft một node, ba partition mặc định và tắt tự tạo topic. run_demo.py tạo topic tường minh trước khi phát, xác nhận consumer group đã gán đủ partition rồi mới bắt đầu producer.

4.2 Trách nhiệm thành phần

Thành phần	Input	Xử lý	Output / kiểm chứng
Simulator / producer.py	rate, count, seed	Sinh nguồn/event, serialize JSON, key theo road, gzip, retry; đếm ACK sau future.get.	Producer log số đã gửi/ACK/lỗi và ACK/s.
Kafka broker	Record từ producer	Lưu topic partitioned theo key; trả metadata/ACK.	Topic, partition, offset, broker health.
consumer.py	Record từ topic	Join group, decode JSON, tính throughput và event-to-consume latency.	Event mẫu, SUMMARY nhận đủ và p50/p95.
run_demo.py	Rate và số lượng benchmark	Tạo topic, chờ consumer, chạy producer, đối chiếu số liệu và ghi log UTF-8.	RESULT PASS hoặc FAIL; log có timestamp.

4.3 Chính sách topic và partition

Topic traffic-events có ba partition và replication factor 1. Khóa record là road_id; record cùng tuyến được đưa vào cùng partition theo partitioner của client, giúp giữ locality/order trong partition. Ba partition minh họa khả năng phân chia luồng và tạo cơ sở để nhiều consumer trong cùng group đọc song song, nhưng thử nghiệm này chỉ chạy một consumer và một broker.

Auto-create topic bị tắt để tránh topic được tạo ngầm với cấu hình mặc định không đúng data contract. AdminClient kiểm tra topic hiện hữu có đúng số partition; nếu không đúng, runner dừng với lỗi cấu hình thay vì âm thầm benchmark sai.

4.4 Độ bền và xử lý lỗi

Cấu hình	Giá trị	Tác dụng / giới hạn
acks	all	Ứng dụng chỉ tính record đã gửi khi Future được xác nhận; một broker/RF=1 không kiểm chứng replication.
retries	10	Thử lại lỗi broker tạm thời; không thay thế xử lý duplicate trong ứng dụng.
linger_ms	5	Cho phép gom batch nhỏ trong producer.
compression_type	gzip	Nén batch khi truyền/lưu; benchmark đo trên tải nén.
auto_offset_reset	latest	Group mới đọc sự kiện phát sau khi group sẵn sàng; runner chờ assignment để không bỏ event đầu.

Kafka mô tả acks=all là chờ các replica in-sync acknowledge; producer retries xử lý các lỗi tạm thời [4]. Trong demo hiện tại, replication factor 1 khiến ACK là xác nhận của một broker duy nhất, không có failover. Không tuyên bố exactly-once hoặc HA.

5. Cài đặt và triển khai

5.1 Cấu trúc mã nguồn

```
docker-compose.yml       # Kafka KRaft m■t broker, volume, healthcheck
requirements.txt         # kafka-python==2.3.3
src/producer.py          # Simulator và publisher
src/consumer.py          # Consumer monitor và ■o s■ li■u
src/run_demo.py          # T■o topic, ph■i h■p benchmark, ghi log
logs/                    # Log benchmark UTF-8 có timestamp
```

5.2 Cấu hình và khởi động

```
docker compose up -d --wait
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python src/run_demo.py --rates 100,500,1000 --count 5000
```

Lượt trên tạo 5.000 event cho mỗi rate, mỗi run dùng consumer group riêng. Script tạo topic nếu chưa tồn tại, xác nhận partition count bằng AdminClient, chờ READY từ consumer rồi chạy producer. Trên máy hiện tại Python được cài ở LOCALAPPDATA\Programs\Python\Python314; nếu launcher py -3 không nhận diện, README có lệnh thay thế theo đường dẫn python.exe.

5.3 Producer

Producer chọn event_type theo tỉ lệ 70/20/10, dựng envelope và payload, tạo event_time theo UTC, serialize JSON UTF-8 và gửi key road_id. Gửi bất đồng bộ theo lô 100; sau flush, chương trình lấy kết quả Future.get() từng record để tách chính xác số gửi khỏi số broker đã xác nhận. Runner coi producer là lỗi nếu ACK không bằng số gửi hoặc có lỗi giao hàng.

```
KafkaProducer(
    bootstrap_servers=...,
    acks="all",
    retries=10,
    linger_ms=5,
    compression_type="gzip",
)
future = producer.send(topic, key=event["road_id"], value=event)
producer.flush()
metadata = future.get(timeout=30)
```

5.4 Consumer và đo lường

Consumer dùng group id riêng theo từng rate, auto_offset_reset=latest, decode value từ JSON và poll theo batch. Trước khi báo READY, poll một lần để join group và nhận assignment; runner không phát dữ liệu trước READY. Consumer đếm toàn bộ record và kết thúc khi đủ count. SUMMARY ghi elapsed, avg_rate, p50/p95 event-to-consume, sample_count và partition count.

Event-to-consume được tính từ thời điểm event_time của simulator đến khi consumer xử lý record. Consumer dùng đồng hồ cùng máy để loại bỏ sai lệch đồng hồ giữa source và broker; trị số không so sánh trực tiếp với hệ thống phân tán đặt ở nhiều máy/region. Với trên 10.000 event, p50/p95 sử dụng tối đa 10.000 mẫu gần nhất.

5.5 Khả năng tái lập

- Docker Compose quản lý phiên bản broker, môi trường KRaft, cổng, volume và healthcheck.
- requirements.txt ghim đúng phiên bản client đã chạy trên máy phát triển.
- run_demo.py ghi tham số, TOPIC_READY, event mẫu, metric và RESULT vào logs/ingestion-<timestamp>.log.
- Seed điều khiển lựa chọn event payload trong từng lượt; mỗi event_id là UUID riêng.
- README.md chứa setup, lệnh chạy/dừng, cách đọc chỉ số, schema và giới hạn.

6. Thực nghiệm và đánh giá

6.1 Môi trường

Hạng mục	Cấu hình thực nghiệm
Máy phát triển	Intel Core i7-1355U, RAM 15,6 GB
Hệ điều hành	Windows 11 build 26200
Container	Docker Compose 5.0.2; một container apache/kafka:4.1.0
Broker	KRaft combined broker/controller, 3 partition/topic, RF=1
Producer/consumer	Python 3.14.5; kafka-python 2.3.3
Topic	traffic-events; key road_id; auto-create off

6.2 Thiết kế kịch bản

Ba mức tải lần lượt là 100, 500 và 1.000 sự kiện/giây. Mỗi mức gửi 5.000 message, tổng cộng 15.000. Consumer group mới được tạo cho mỗi lượt, topic và broker được giữ cố định. Producer pacing dùng monotonic clock; consumer nhận count hữu hạn để có thể đối chiếu số lượng đầu vào/đầu ra. Kết quả đo là một lần chạy mỗi mức trên máy cục bộ, chưa phải benchmark thống kê nhiều lần.

Kịch bản	Mục tiêu	Số event	Kết quả cần đạt
L1	100 event/s	5.000	5.000 ACK và 5.000 consumer receive.
L2	500 event/s	5.000	5.000 ACK và 5.000 consumer receive.
L3	1.000 event/s	5.000	5.000 ACK và 5.000 consumer receive.

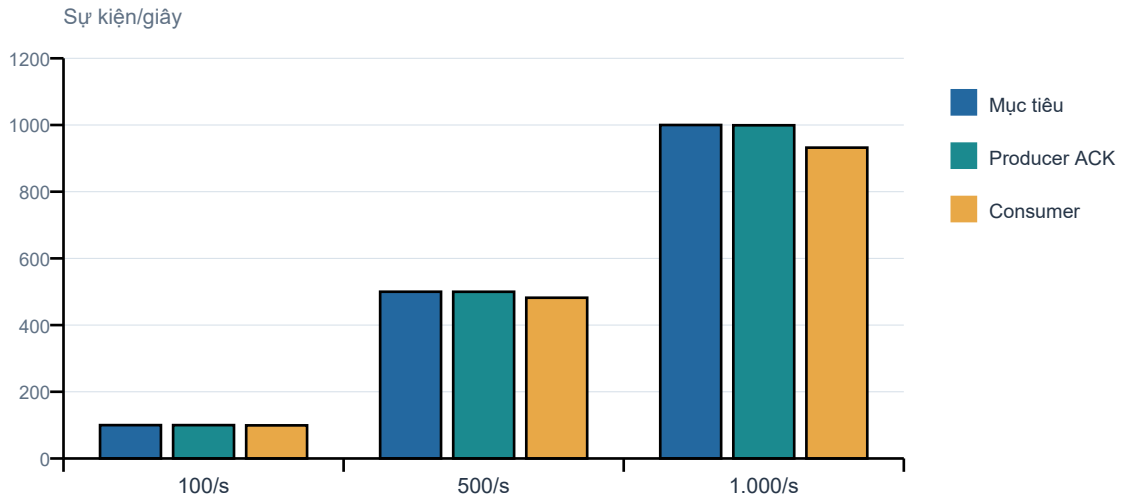
6.3 Kết quả định lượng

Rate đặt	Producer ACK rate	Consumer avg throughput	p50	p95	ACK / receive
100/s	100,0/s	99,3/s	11,2 ms	18,1 ms	5.000 / 5.000
500/s	499,9/s	482,0/s	10,1 ms	19,9 ms	5.000 / 5.000
1.000/s	999,2/s	932,1/s	11,9 ms	20,7 ms	5.000 / 5.000

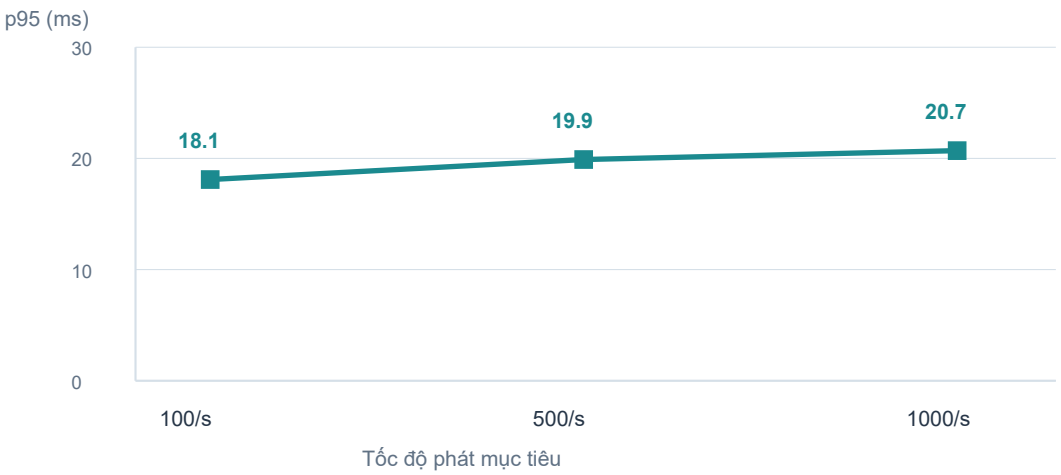
Nguồn: log logs/ingestion-20261031-051334.log. Throughput consumer được tính bằng received / elapsed trong SUMMARY; p50/p95 là phân vị trên 5.000 event-to-consume latency mẫu của từng lượt. Producer ACK rate được tính từ số future đã xác nhận chia thời gian chạy producer. Do consumer còn thời gian startup/poll và máy chạy broker + client cùng lúc, avg throughput consumer có thể thấp hơn rate mục tiêu/ACK rate.

Phân bố event_type ghi nhận ở consumer

Loại event	Số nhận gộp 3 lượt	Tỷ trọng
gps_position	10.508	70,1%
road_sensor	3.030	20,2%
traffic_incident	1.462	9,7%
Tổng	15.000	100,0%



Hình 2. So sánh tốc độ phát mục tiêu, broker ACK và throughput consumer.



Hình 3. p95 độ trễ event-to-consume theo từng mức tải. Đường tăng nhẹ từ 18,1 ms lên 20,7 ms trong phạm vi phép đo cục bộ.

6.4 Đánh giá theo yêu cầu

Tiêu chí	Bằng chứng	Đánh giá
Pipeline hoạt động	TOPIC_READY; producer ACK=5.000; consumer SUMMARY=5.000; ID/sequence không trùng hoặc khuyết.	Đạt cho phạm vi ingestion.
Thiết kế và cài đặt	Compose KRaft, topic explicit, schema envelope/payload, key, group và ACK policy.	Đạt; cấu hình phù hợp demo cục bộ.
Thực nghiệm	Ba tải; log ghi rate, thời gian, ACK, receive, p50/p95.	Đạt kiểm chứng; cần lặp nhiều lần nếu cần độ tin cậy thống kê.
Big Data / mở rộng	15.000 sự kiện tổng, tốc độ tới 1.000/s, ba partition và nhiều loại event.	Có minh chứng velocity/variety; chưa chứng minh scale-out cluster.
Tái lập	README, requirements ghim phiên bản, Compose, seed, runner và log.	Đạt trên môi trường có Docker/Python.

7. Kết quả demo

7.1 Luồng demo

Khởi động Kafka bằng Docker Compose; runner tạo/kiểm tra topic ba partition; consumer join group và báo READY; producer phát event theo từng rate; broker xác nhận; consumer in event mẫu, metric cửa sổ và SUMMARY; runner đối chiếu ACK, số record, tính duy nhất của ID/sequence, khoảng sequence khuyết và đủ event_type rồi ghi RESULT PASS. Log benchmark đầy đủ được lưu trong thư mục logs để giảng viên có thể kiểm tra từng bước mà không phụ thuộc ảnh dashboard.

```
TOPIC_READY name=traffic-events | partitions=3 | replicas=1
CONSUMER READY ... | partitions=3
PRODUCER Producer stopped | sent=5000 | acknowledged=5000 | failed=0
CONSUMER SUMMARY received=5000 | ... | event_to_consume_p95=...
RESULT PASS | ACKs, counts, IDs, sequences, and event types verified
```

7.2 Minh chứng nộp

- logs/ingestion-20261031-051334.log: log UTF-8 của ba mức tải; gồm topic, partition, mẫu record, kiểm tra integrity, phân bố event_type, metric và kết quả pass.
- README.md: lệnh chạy tái lập và hướng dẫn lấy docker compose ps / topic description khi trình bày trực tiếp.
- Mã nguồn và docker-compose.yml: cấu hình, simulator, producer, consumer, runner và schema.

Log là minh chứng đủ để kiểm tra record count và số đo của quy trình 1. Ảnh chụp màn hình hoặc video có thể bổ sung cho phần trình bày nhưng không thay thế log và mã nguồn.

8. Thảo luận

8.1 Điểm đạt được

- Một lệnh benchmark tự tạo topic và chờ consumer sẵn sàng, giảm race condition mất message đầu.
- Tách số record đưa vào producer buffer khỏi số broker ACK; runner yêu cầu số consumer nhận bằng count.
- Hợp đồng envelope + payload cho ba loại nguồn giúp thể hiện variety trong cùng topic.
- Ba tải tạo throughput tăng gần tuyến tính đến khoảng 1.000/s trên máy được thử; p95 trong khoảng 18-23 ms ở các lượt này.

8.2 Hạn chế

- Dữ liệu là giả lập, không phản ánh phân phối, tần suất hoặc tính chính xác của giao thông thực.

- Broker duy nhất và replication factor 1 không có HA, không kiểm thử mất node và không chứng minh mở rộng ngang thực tế.
- Tổng 15.000 event là tải benchmark nhỏ; cần chạy dài hơn/khối lượng lớn hơn trên cluster để kết luận quy mô production.
- Mỗi rate chỉ chạy một lần; không có warm-up riêng, nhiều vòng lặp, khoảng tin cậy hay kiểm soát nhiễu hệ thống.
- p95 đồng hồ một máy; đo trên nhiều máy cần đồng bộ thời gian hoặc dùng latency instrumentation riêng.
- Không triển khai schema registry, dead-letter queue, authentication/TLS, observability dashboard hoặc lưu data lake; các mục này ngoài phạm vi.

8.3 Hướng cải tiến trong pipeline tiếp theo

Một nhóm downstream có thể đọc traffic-events bằng consumer group riêng, ghi dữ liệu gốc vào object storage/Data Lake, xử lý cửa sổ bằng Spark Structured Streaming và phân tích điểm ùn tắc. Giai đoạn đó cần tiếp tục xác định event-time, watermark, chất lượng dữ liệu, checkpoint và cơ chế xử lý dữ liệu đến muộn; không thay đổi ranh giới đã chọn của đồ án này.

8.4 Phân công nhóm

Thông tin thành viên và phân công phải điền theo đóng góp thực tế; không suy diễn từ người viết báo cáo hoặc công cụ hỗ trợ.

Thành viên	Mã số SV	Phần việc thực hiện	Minh chứng
Bổ sung	Bổ sung	Bổ sung theo nhóm	Commit / module / phần trình bày
Bổ sung	Bổ sung	Bổ sung theo nhóm	Commit / module / phần trình bày
Bổ sung	Bổ sung	Bổ sung theo nhóm	Commit / module / phần trình bày
Bổ sung	Bổ sung	Bổ sung theo nhóm	Commit / module / phần trình bày
Bổ sung	Bổ sung	Bổ sung theo nhóm	Commit / module / phần trình bày

9. Kết luận

Đồ án đã triển khai và kiểm chứng quy trình Sources -> Ingestion bằng Python và Apache Kafka. Producer mô phỏng ba loại nguồn giao thông, gửi JSON với khóa theo tuyến đường vào topic ba partition, broker xác nhận từng record, consumer đọc đủ và đo throughput/latency. Trong ba thử nghiệm 5.000 sự kiện ở 100, 500 và 1.000 event/s, số ACK khớp số nhận; event_id/sequence không trùng hoặc khuyết; throughput consumer lần lượt 99,3; 482,0; 932,1 event/s; p95 latency 18,1; 19,9; 20,7 ms.

Sản phẩm đáp ứng quy trình ingestion và có bằng chứng tái lập. Kết luận về scale cần giới hạn ở tải đo cục bộ: dữ liệu mô phỏng, một broker, replication factor 1. Bài làm không tuyên bố đã dựng một hệ thống Big Data production hay chứng minh khả năng chịu lỗi/HA.

Kết quả này có thể làm đầu vào cho giai đoạn lưu trữ và xử lý tiếp theo, nhưng mỗi giai đoạn cần được đánh giá riêng để không nhầm thành phần downstream với phạm vi mà nhóm đã thực sự triển khai.

Phụ lục A. Đối chiếu yêu cầu đồ án

Yêu cầu đề bài	Vị trí / bằng chứng
Chọn quy trình và ranh giới	Mục 1; Sources -> Ingestion, kết thúc tại topic.
Ít nhất hai đặc trưng Big Data	Mục 2: velocity, variety; volume dùng làm tải thử và nêu giới hạn.
Input, format, schema, ví dụ	Mục 3; schema envelope/payload, sample JSON, generator.
Output chính xác	Mục 3.5; Kafka topic, key, value, partition/offset.
Công nghệ và vai trò	Mục 2.4 và 4.2-4.4.
Sơ đồ data-flow	Mục 4.1.
Demo và số liệu	Mục 6-7; log benchmark ba tải, ACK/receive, throughput, p50/p95.
Khả năng tái lập	Mục 5.2, README, Compose, requirements, runner và generator.
Mã nguồn/cấu hình/log	Thư mục dự án QuyTrinh1-Sources-Ingestion.
Phân công và đóng góp	Mục 8.4 - nhóm phải điền theo thực tế.

Đề bài khuyến nghị báo cáo khoảng 30 trang, không tính trang bìa, mục lục, tài liệu tham khảo và phụ lục; độ dài không thay thế chất lượng nội dung, tính trung thực và bằng chứng chạy thật [1].

Phụ lục B. Tài liệu tham khảo

[1] Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn. Yêu cầu đồ án học phần Nhập môn dữ liệu lớn, 2026. Tài liệu do học phần cung cấp.

[2] Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn. Slide Chương 1 - Giới thiệu dữ liệu lớn; Slide Chương 6 - Streaming và xử lý dữ liệu luồng, 2026.

[3] Apache Kafka. Quickstart - Apache Kafka 4.1. Tạo topic, producer/consumer, mô hình event và partition. https://kafka.apache.org/41/getting-started/quickstart/

[4] Apache Kafka. Producer Configs - Apache Kafka 4.1. Mô tả acks, retries, batching và compression. https://kafka.apache.org/41/configuration/producer-configs/

[5] Apache Kafka. Downloads - Apache Kafka. Release và Docker image 4.1.0. https://kafka.apache.org/community/downloads/

[6] kafka-python. API Documentation. Producer, AdminClient, Consumer và FutureRecordMetadata. https://kafka-python.readthedocs.io/en/stable/

[7] Apache Kafka. Docker - Apache Kafka 4.1. Cấu hình và chạy Docker image. https://kafka.apache.org/41/getting-started/docker/

Các trang web được truy cập ngày 31/10/2026. Tham chiếu kỹ thuật dùng để giải thích thiết kế; kết quả benchmark lấy trực tiếp từ log chạy của dự án.

Phụ lục C. Công cụ AI và xác nhận kỹ thuật

OpenAI Codex được dùng để hỗ trợ phân tích yêu cầu, dựng và chỉnh mã nguồn, khắc phục lỗi encoding/benchmark, và soạn bản thảo báo cáo. Các con số trong báo cáo được trích từ lần chạy thực tế có log; mã nguồn và luồng được chạy trên Kafka container. Nhóm nộp bài cần tự đọc, hiểu và chịu trách nhiệm về từng quyết định kỹ thuật, kiểm tra lại thông tin thành viên/phân công, và chỉnh nội dung báo cáo cho khớp phần việc nhóm thực sự đã làm theo quy định học thuật [1].